Proyecto: Pub

Tercera parte

En esta tercera parte construiremos en el paquete `packPub` la clase `PubManagement`, que contendrá el programa principal de la aplicación, desde el que se comenzará la ejecución de las distintas opciones para la gestión del Pub.

El programa principal comenzará dando la bienvenida al usuario y, a continuación, mostrará un menú desde el que se podrán seleccionar las operaciones de implementadas en nuestra aplicación. El menú debe contener todas las opciones que se han ido describiendo en el apartado "Tareas a realizar" de las partes anteriores del proyecto. Es necesario controlar que el usuario elija opciones válidas incluidas en el menú para que no se eleven excepciones durante la entrada de datos. Por otro lado, tal vez sea adecuado utilizar métodos auxiliares para conseguir una implementación más limpia y fácil de testear.

Cambiar el método `toString` de la clase `Food` para que forme y devuelva un String con los valores de todos los atributos, utilizando el símbolo | en vez del espacio en blanco para separar la información de los distintos atributos. Además, en esa misma clase hay que tener en cuenta que las calorías no pueden ser un valor negativo, por lo que si se intentara asignar a ese atributo un valor negativo, debería elevarse la excepción *`IllegalArgumentException`*.

También debes añadir los siguientes métodos en la clase pub:

- `addFood`. Recibe un objeto tipo food y lo añade a la lista de comidas. Este método deberá controlar que todos los elementos de la lista de alimentos sean diferentes (se considera que dos alimentos son iguales cuando tienen el mismo nombre). Al intentar introducir uno repetido, debe elevarse la excepción *`IllegalArgumentException`*.
- `storeFoodInFile`**.** Almacena todas las comidas de la lista de comidas del pub en un fichero con el formato devuelto por el método `toString`. El fichero se generará siempre dentro de la carpeta **data**. El nombre de cada nuevo fichero debe introducirse utilizando el método `showInputDialog` de la clase `JoptionPane`. Este método no debe propagar excepciones (`IOException` o `NullPointerException`) y debe crear siempre el fichero con el nombre indicado por el usuario.
- `loadFoodFromFile`. Lee el fichero que tiene el nombre indicado por el usuario, a través de la ventana que se muestra utilizando el método `showInputDialog` de la clase `JoptionPane`. Añade en la lista de `Food` contenida en `Pub`, los datos de las comidas leídos del fichero. Si una línea del fichero está defectuosa o la comida ya está en la lista, debe continuar leyendo la siguiente comida.

La clase `pubManagement` tiene un atributo con la referencia a la bolsa de pedidos. En nuestro `Pub` se pueden hacer pedidos para enviar. Para la bolsa de pedidos (que es única) tendremos una clase llamada `OrderSet` dentro del paquete de `packPub`. Esta clase guarda una listas de pedidos (`Orderlist`). Cada pedido (`Order`) contará con un identificador (que se creará de forma automática) y una lista de elementos que son enviables.

La clase `OrderSet` tiene un método para añadir un pedido al final de la lista. Un segundo método devolverá el primer pedido de la lista eliminándolo de la misma.

Tareas a realizar:

1. Completar el diagrama de clases UML de la segunda parte del Proyecto, incluyendo las clases y métodos de la tercera parte..
2. Implementa y documenta el código correspondiente a todo lo previamente indicado
3. Incluir en el menú del método main de la clase PubManagement las siguientes opciones:

   3.1. Incorporar un nuevo pedido.

   3.2. Borrar un pedido y mostrarlo por pantalla.

   3.3. Guardar la comida en un archive.

   3.4. Leer la comida desde el archivo.

   3.5. Añadir dos opciones más que consideres oportunas